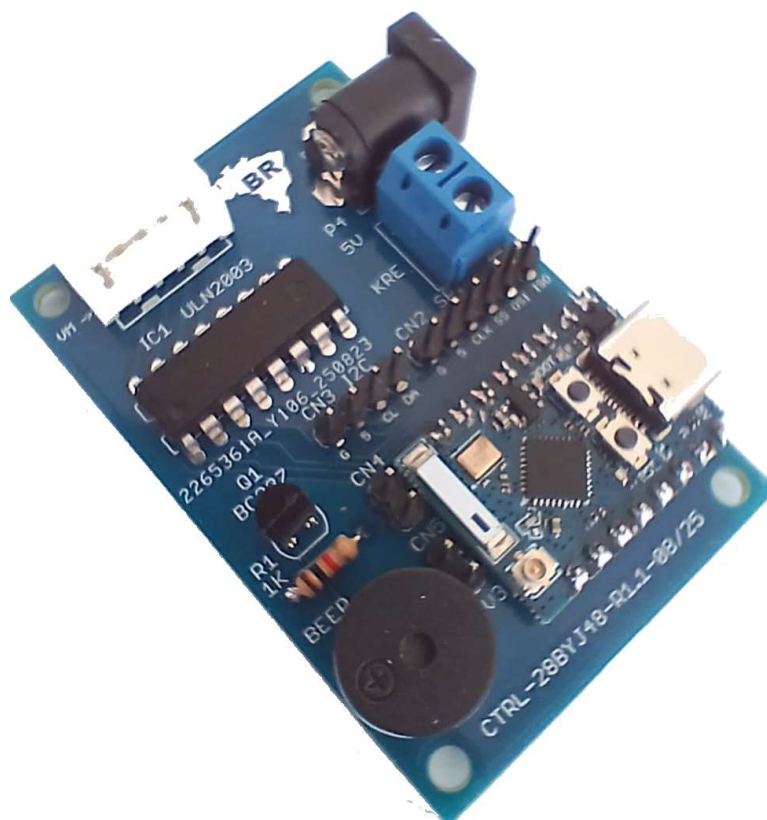


PCI CTRL-28BYJ48



MANUAL DO USUÁRIO

Aplicável as placas:

CTRL-28BYJ48 versão 1.1 e posteriores
com o módulo microcontrolador ESP32 C3 supermini de 16 pinos

Manual rev. 1.2 — 06/2025

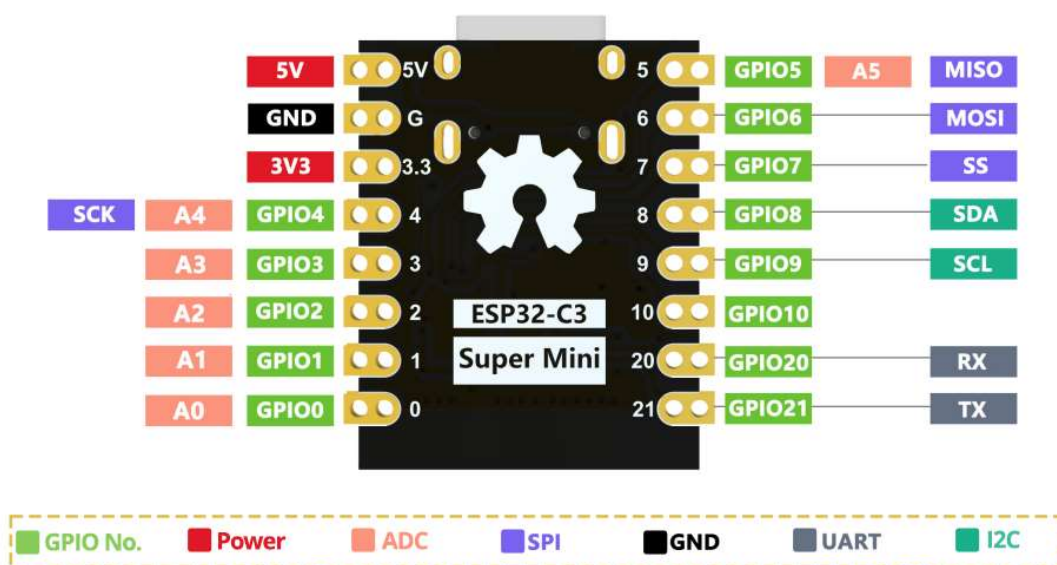
Eventuais atualizações desse manual podem ser encontradas em: <https://github.com/izyino/manuais>

INTRODUÇÃO:

A presente placa de circuito impresso intitulada CTRL-28BYJ48 foi concebida para facilitar a construção de dispositivos para controle de motor de passo do tipo 28BYJ48 via WiFi, contemplando em um reduzido espaço de apenas 4 x 6 cm, o moderno microcontrolador ESP32 C3 em sua versão supermini de 16 pinos, o driver do motor de passo e todos os demais componentes, substituindo assim os antigos, obsoletos e saudosos Arduinos e seus “shields” sobrepostos em forma de sanduíche.

A versão 1.1 da placa de circuito impresso CTRL-28BYJ48 possui as seguintes características básicas:

- Utilização do microcontrolador ESP-32 C3 na versão supermini de 16 pinos, com WiFi e Bluetooth, CPU de 32-bits RISC-V Single-core 160MHz, WiFi: 802.11b/g/n 2.4GHz, Bluetooth 5.0, Consumo ultra baixo de apenas 43uA, 400KB SRAM, 384KB ROM, 4Mb flash (pinagem e sinais disponíveis na figura abaixo)
- Controle de dispositivos com base em motor de passo modelo 28BYJ48 ou similar
- Utilização do driver ULN2003
- Beep sonoro passivo
- Conectores para acesso aos sinais I2C, SPI, Tx/Rx
- Completa biblioteca para controle do motor de passo, do beep sonoro e do Led azul, bem como um controle de tempo, não bloqueante, em alternativa às funções millis() e delay(), disponível em: <https://github.com/izyino/motbepled>



CONECTORES EXISTENTES NA PLACA CTRL-28BYJ48:

CN1 – 28BYJ48:

- 1 – 5Vcc (fio vermelho)
- 2 – L1 (fio laranja)
- 3 – L1 (fio amarelo)
- 4 – L2 (fio rosa)
- 5 – L2 (fio azul)

CN2 – SPI:

- 1 – Gnd
- 2 – Vcc (+5V)
- 3 – SCK (GPIO 4 do ESP32C3)
- 4 – SS (GPIO 7 do ESP32C3)
- 5 – MOSI (GPIO 6 do ESP32C3)
- 6 – MISO (GPIO 5 do ESP32C3)

CN3 – I2C:

- 1 – Gnd
- 2 – Vcc (+5V)
- 3 – SCL (GPIO 9 do ESP32C3)
- 4 – SDA (GPIO 8 do ESP32C3)

CN4 – Tx Rx:

- 1 – Tx (GPIO 21 do ESP32C3)
- 2 – Rx (GPIO 20 do ESP32C3)

CN5 – 3V3:

- 1 – Gnd
- 2 – +3V3

RELAÇÃO DE COMPONENTES:

- Placa de circuito impresso Rev.1.1
- Módulo ESP32 C3 supermini, 16 pinos
- CI driver ULN2003, dip 16
- Conector molex 5 pinos com polarizador (CN1, para o motor)
- Conector P4 fêmea, 90 graus, solda placa
- Borne KRE de 2 pinos (montagem opcional)
- Beep passivo (montagem opcional)
- Transistor BC337 (montagem opcional)
- Resistor de 1K montagem opcional)
- Conector KRE de 2 pinos (montagem opcional)
- Conector 6 pinos macho (CN2 – SPI, montagem opcional)
- Conector 4 pinos macho (CN3 – I2C, montagem opcional)
- Conector 2 pinos macho (CN4 – TxRx, montagem opcional)
- Conector 2 pinos macho (CN5 – 3V3, montagem opcional)

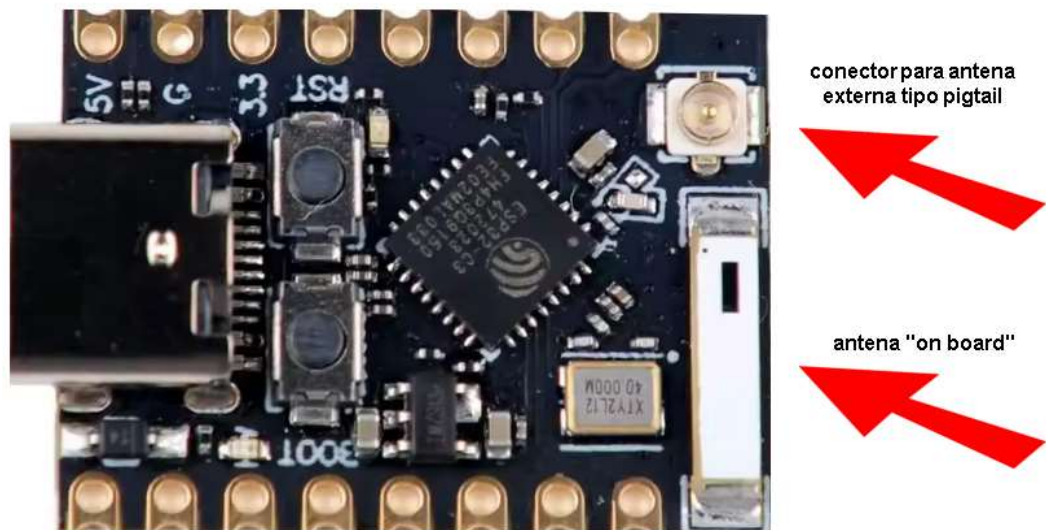


Fig. 1 – Modelo de ESP32 C3 recomendado

Existe diversos modelos diferentes de ESP32 C3 disponíveis no mercado. O modelo que mostrou mais facilidade de conexão e melhor desempenho, especialmente no tocante ao alcance WiFi é o mostrado na figura 1 acima.

O modelo mais comum, mostrado na figura 2 abaixo, **deve ser evitado** uma vez que esse modelo possui um alcance muito menor em virtude da sua antena "on board" ser visivelmente menos elaborada, sem contar com a ausência do conector para antena externa.

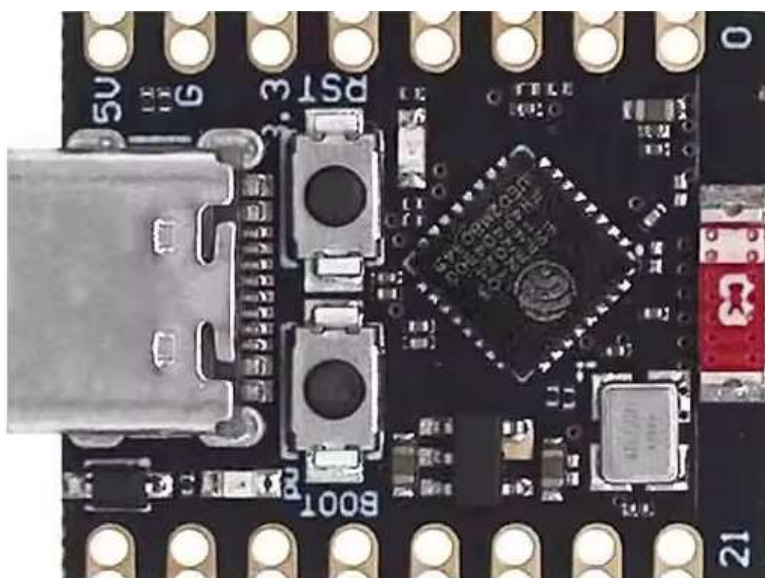


Fig. 2 – Modelo de ESP32 C3 **-NÃO-** recomendado

DIAGRAMA ELÉTRICO:

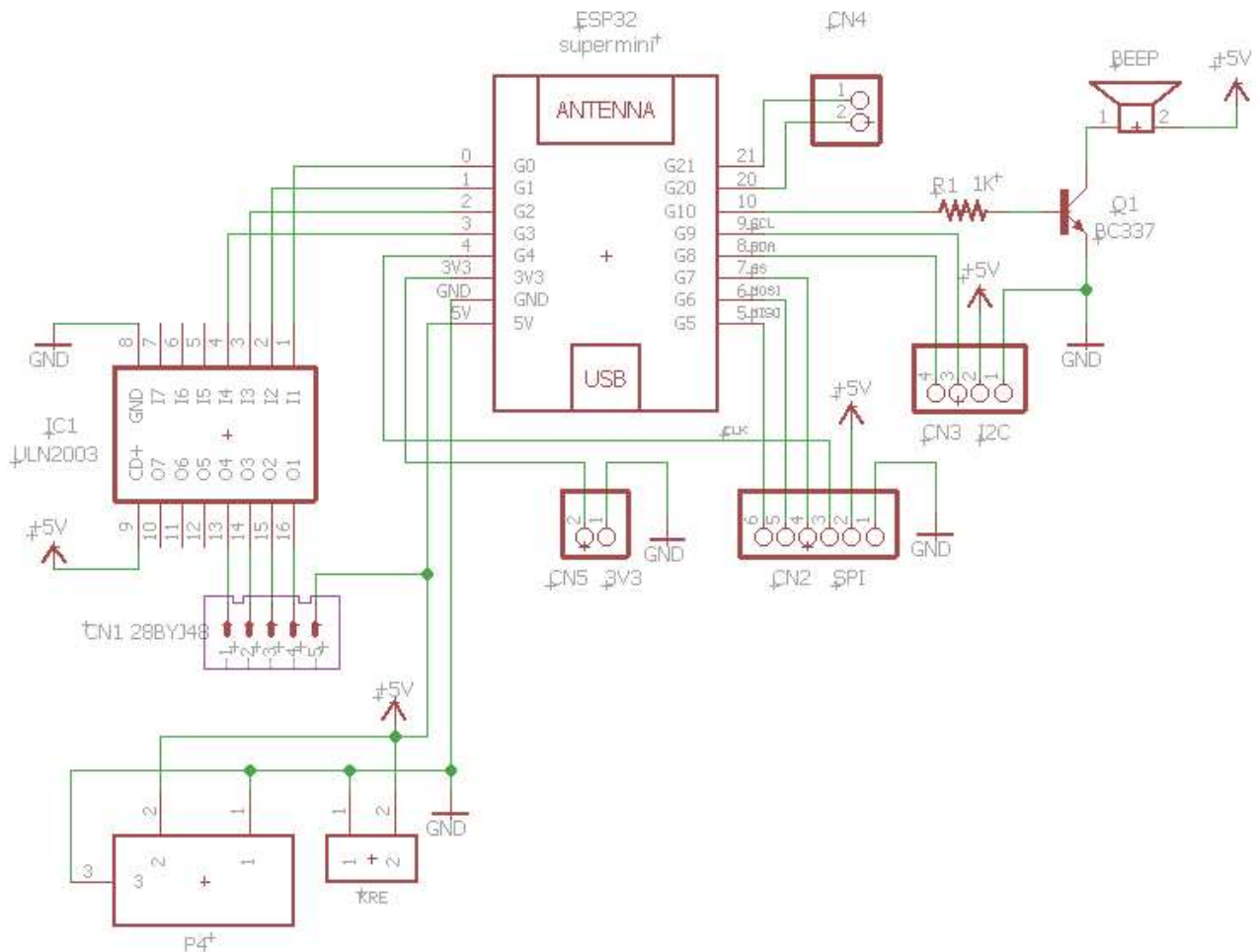


Fig. 3 – Diagrama elétrico da placa CTRL-28BYJ48 Rev. 1.1

IMPORTANTE: Algumas placas da revisão 1.1 possuem o pino 5 do CN1 (28BYJ48) sem nenhuma conexão, e foram deixados em aberto. Nesses casos é necessário que se ligue esse pino 5 do CN1 ao Vcc (+5V) para que os motores 28BYJ48 funcionem corretamente

DIMENSÕES DA PLACA E CONECTORES:

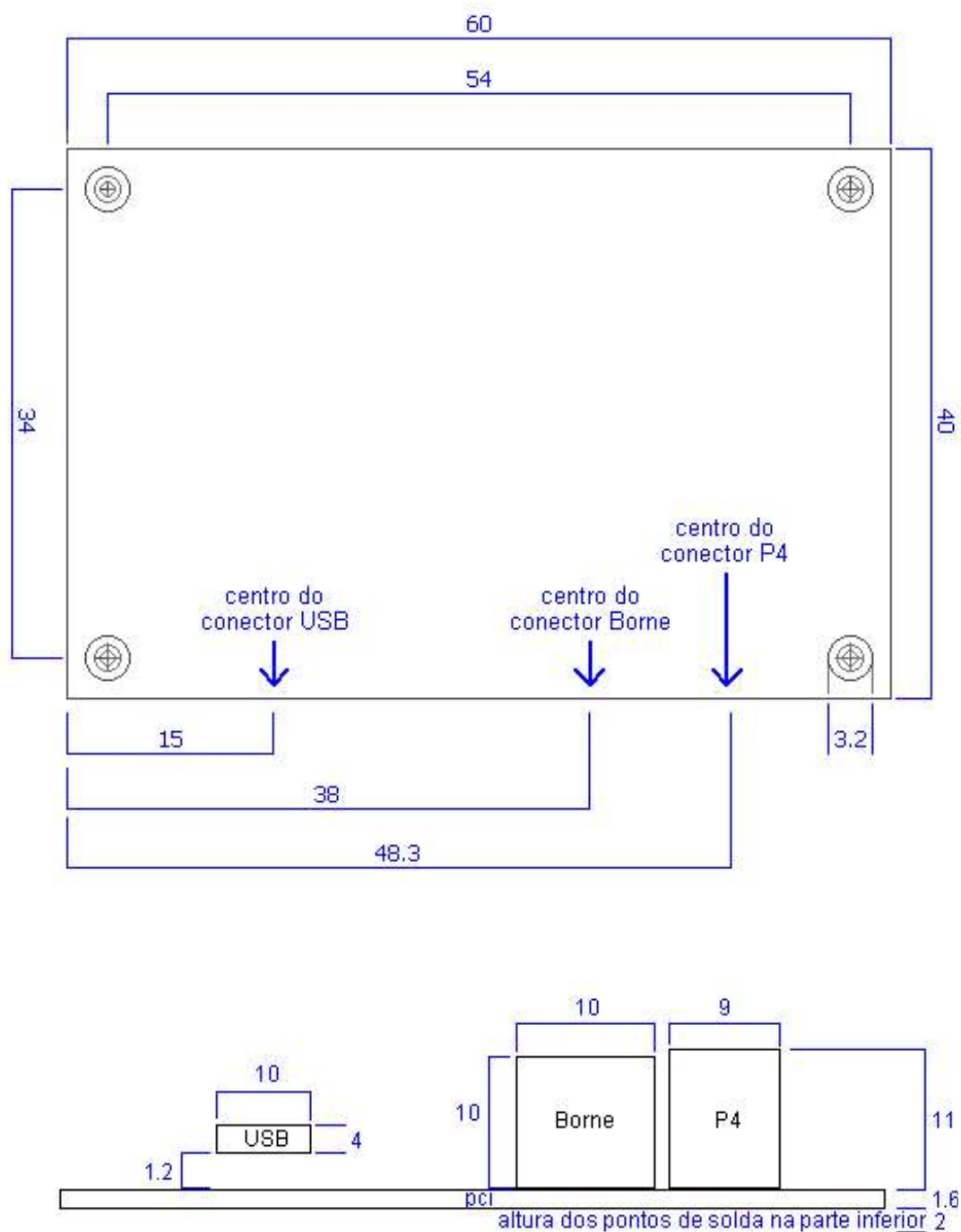


Fig. 4 – Dimensões da placa e conectores

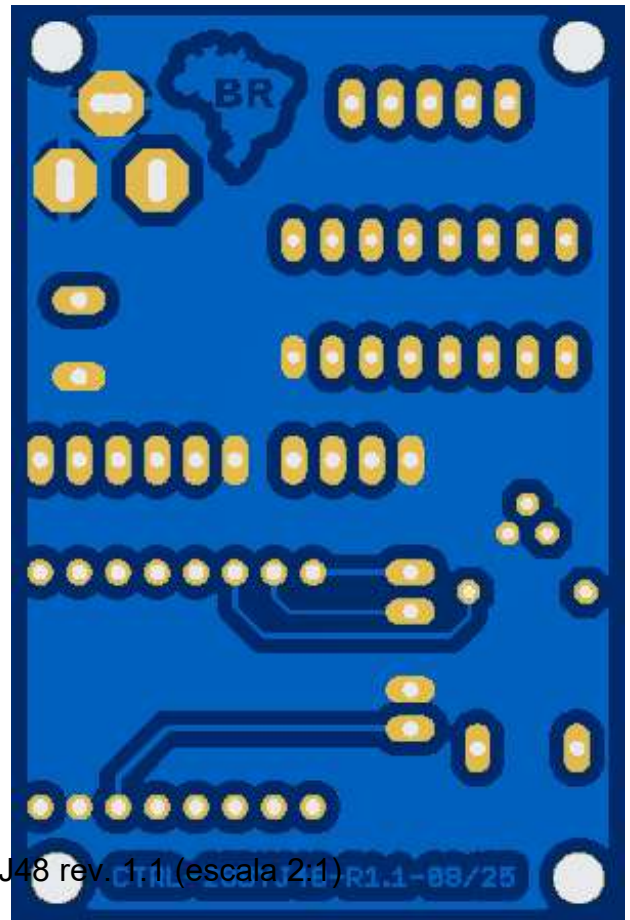
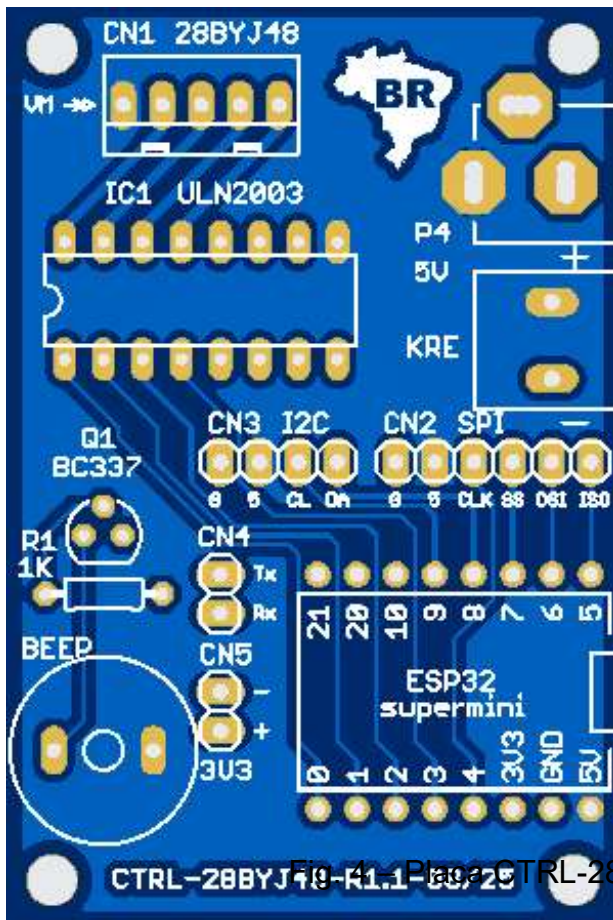


Fig. 41 – Placa CTRL-28BYJ48 rev. 1.1 (escala 2:1)

BIBLIOTECA:

Para facilitar o desenvolvimento de aplicações a serem hospedadas em placas baseadas nos microcontroladores ESP32, foi criada uma biblioteca de nome `motbepled.h`, disponível em <https://github.com/izyino/motbepled> a qual deve ser utilizada conforme as instruções seguintes.

As funções descritas a seguir referem-se apenas àquelas relevantes para a placa CTRL_28BYJ48. Registre-se que a biblioteca `motbedled.h` possui muitas outras funções que fazem sentido apenas para placas mais complexas que a CTRL_28BYJ48.

`#include < motbepled.h>`

para incluir a biblioteca ao programa. Dependendo de onde a biblioteca estiver gravada, pode-se usar alternativamente o formato `#include "motbepled.h"`

motbepled x(t);

comando construtor que deve ser informado logo após o include, sendo t uma variável do tipo int8_t que define o tipo e o modo de operação do motor conectado a CN1, sendo possível os seguintes valores:

- 1 – Para motor 28byj-48, 2048 passos por volta, baixo torque, baixo consumo
- 2 – Para motor 28byj-48, 2048 passos por volta, alto torque, alto consumo
- 3 – Para motor 28byj-48, 4096 passos por volta, médio torque, médio consumo

x.pinsStep0(0, 1, 2, 3, -1, -1);

comando que informa os pinos do microcontrolador ESP associados ao motor de passo, sendo os quatro primeiros das bobinas principais e os dois últimos dos sinais enable. O valor -1 significa que os sinais correspondentes não se aplicam no presente caso. Para uso da biblioteca motbepled.h com a placa CTRL_28BYJ48 deve-se informar o comando pinsStep0 exatamente como mostrado acima, ou seja: x.pinsStep0 (0, 1, 2, 3, -1, -1); e deve ser informado na sessão de setup de todos os programas, sempre antes do x.begin()

x.pinBeep(10);

comando que informa o pino do microcontrolador ESP associado ao beep. Para uso da biblioteca motbepled.h com a placa CTRL_28BYJ48 deve-se informar o comando pinBeep exatamente como mostrado acima, ou seja: x.pinBeep (10); e deve ser informado na sessão de setup de todos os programas, sempre antes do x.begin()

x.pinLed(8, 0);

comando que informa o pino do microcontrolador ESP associado ao Led e o nível lógico para o led aceso. Para uso da biblioteca motbepled.h com a placa CTRL_28BYJ48 deve-se informar o comando pinLed exatamente como mostrado acima, ou seja: x.pinLed (8, 0); e deve ser informado na sessão de setup de todos os programas, sempre antes do x.begin()

x.begin();

inicializa as diversas funções da biblioteca. Deve ser colocado na sessão de setup de todos os programas que se utilizem da biblioteca

x.runStep(0, steps, velstep, cwstep);

comando que ativa o motor de passo, de forma automática e assíncrona, conforme as seguintes variáveis:

steps – variável uint32_t contendo o número de passos a movimentar

velstep – variável uint16_t que define a velocidade da movimentação em RPM (rotações por minuto). Este valor pode variar entre 1 e 16, dependendo do motor utilizado e da corrente disponível na fonte de alimentação

cwstep – variável booleana que define o sentido da movimentação, sendo “true” para sentido horário e “false” para sentido anti-horário

x.stepstogo(0);

esta função retorna no formato uint32_t o número de passos ainda restantes para que o motor chegue ao seu destino. Zero significa que o motor já chegou ao seu último destino e já encontra-se parado. Antes de comandar qualquer movimentação deve-se consultar esta função para ter certeza que o motor encontra-se parado

x.runStepAcc(0, steps, velini, velmax, accel, cwstep);

comando que ativa o motor de passo, de forma automática e assíncrona, **com aceleração de desaceleração**, conforme as seguintes variáveis:

steps – variável uint32_t contendo o número de passos a movimentar

velini – variável do tipo **float** que define a velocidade inicial da movimentação em RPM (rotações por minuto)

velmax – variável do tipo **float** que define a velocidade máxima da movimentação em RPM (rotações por minuto)

accel – variável do tipo **float** que define a aceleração, em passos/s²

cwstep – variável booleana que define o sentido da movimentação, sendo “true” para sentido horário e “false” para sentido anti-horário

x.stepstogoacc(0);

esta função retorna no formato uint32_t o número de passos ainda restantes para que o motor chegue ao seu destino, após execução do comando x.runStepAcc. Zero significa que o motor já chegou ao seu último destino e já encontra-se parado. Antes de comandar qualquer movimentação deve-se consultar esta função para ter certeza que o motor encontra-se parado

x.beep(bnum, bdur, bfreq, binter);

comando que ativa a emissão de beeps sonoros, de forma automática e assíncrona, conforme as seguintes variáveis:

bnum – variável inteira que especifica o número de beeps a serem emitidos

bdur – variável inteira que especifica a duração de cada beep, em milisegundos

bfreq – variável inteira que especifica a frequência dos beeps, em Hertz (Hz). Os beeps passivos comuns respondem bem frequências entre 200Hz e 5000Hz

binter – variável inteira que especifica a duração da pausa entre os beeps, em milisegundos

x.led(lnum, ldur, linter);

comando que ativa piscadas do Led (conectado ao pino 8 do módulo ESP32) , de forma automática e assíncrona, conforme as seguintes variáveis:

lnum – variável inteira que especifica o número de piscadas a serem emitidas

ldur – variável inteira que especifica a duração do Led acesso em cada piscada, em milisegundos

linter – variável inteira que especifica a duração do Led apagado em cada piscada, em milisegundos

x.setms(yms);

comando para inicializar o contador de milisegundos com o valor informado pela variável yms do tipo uint32_t. Imediatamente após inicializado, o contador começa ser subtraído de 1 a cada milisegundo

x.getms();

esta função retorna no formato uint32_t o estado atual do contador de milisegundos previamente inicializado pelo comando x.setms. Serve como alternativa para a função delay(), de forma assíncrona

x.stopStep(0);

esta função interrompe o movimento do motor de passo, após a execução do comando x.runStep

x.stopStepAcc(0);

esta função interrompe o movimento do motor de passo, após a execução do comando x.runStepAcc

x.stopLed();

esta função interrompe as piscadas do Led eventualmente em andamento

x.stopBeep();

esta função interrompe a emissão de beeps sonoros eventualmente em andamento

Exemplos de utilização da biblioteca

No início do programa:

```
#include < motbepled.h>
motbepled x(2);
```

na sessão do setup:

```
x.begin();
```

**//movimenta o motor de passo (conectado em CN1), tipo 28BYJ-48,
//velocidade 3 RPM, sentido horário, 2048 passos:**

```
//função principal:
x.runStep(0, 2048, 3, true);
//o motor começa a se movimentar imediatamente após a função runStep ser chamada
//para saber se o motor de passo já chegou ao destino, fazer
if (x.stepstogo(0)>0) {ainda não chegou ao destino. Está em movimento...};
//a qualquer momento o movimento do motor de passo pode ser interrompido
x.stopStep(0);
```

//emite 10 beeps de 2KHz de 0,5s com pausa interbeeps de 0,25s:

```
//função principal:
x.beep(10, 500, 2000, 250);
//os beeps começam a ser emitidos imediatamente após a função beep ser chamada
//a qualquer momento a emissão dos beeps sonoros pode ser interrompida
x.stopBeep();
```

//pisca o Led 50 vezes com 0,25s aceso seguido de 0,10s apagado:

```
//função principal:
x.led(50, 250, 100);
//o led começa a piscar imediatamente após a função led ser chamada
//a qualquer momento as piscadas do Led podem ser interrompidas
x.stopLed();
```

//contagem de 4 segundos, de forma assíncrona:

//função principal:

x.setms(4000);

while (x.getms()>0){enquanto espera 4s, pode fazer coisas...}

//a variável x.xms começa a ser decrementada imediatamente após ter sido inicializada

O diretório “examples” da biblioteca motbepled contém diversos exemplos de programas, das mais variadas aplicações, como: movimentação de motores, emissão de beeps, acesso a redes WiFi, servidor web, e muitas outras.

IMPORTANTE: Antes da execução de qualquer um dos exemplos, deve-se conferir e alterar quando necessário o tipo e a pinagem dos motores, beep e Led atribuída pelos comandos motbepled, pinsStep, pinsDC, pinBeep e pinLed, para que correspondam fielmente ao hardware utilizado
